Here are the concise, exam-ready 2-mark answers for these internal test questions, formatted for maximum marks:

Question 1:

> **Explain how friend functions are used to overload arithmetic operators in C++.** *(CO1, Remember)*

- **Answer:** A friend function is a non-member function granted access to a class's private/protected members. When overloading binary arithmetic operators (like `+` or `-`) where the left-hand operand might not be an object of your class (e.g., `int + Object`), a friend function allows both operands to be passed explicitly as parameters.
- **Example Syntax:**

```
friend Complex operator+(const Complex& c1, const Complex& c2);
```

Question 2:

> **Differentiate between association and aggregation in object-oriented programming with respect to object relationships.** *(CO1, Understand)*

- **Answer:**
  - **Association:** A general "uses-a" or peer-to-peer relationship where two separate objects are independent and can exist without each other (e.g., a *Car* and a *Driver*).
  - **Aggregation:** A specialized, stronger "has-a" relationship representing whole-part structure where the child object can exist independently of the parent container object (e.g., a *Department* has *Professors*; if the department is closed, the professors still exist).

Question 3:

> **Write a simple C++ class hierarchy to represent a Raspberry Pi-based sensor system, where a base class `Sensor` is inherited by a derived class `TemperatureSensor`. Include constructors and show how they are called in the right order.** *(CO2, Apply)*

- **Answer:**

```
#include <iostream>
using namespace std;

class Sensor {
public:
    Sensor() { cout << "Sensor Base Constructor Called\n"; }
};

class TemperatureSensor : public Sensor {
public:
```

```
    TemperatureSensor() : Sensor() { cout << "TemperatureSensor Derived
Constructor Called\n"; }
};

int main() {
    TemperatureSensor ts; // Output: Base constructor runs first, then derived.
    return 0;
}
```

- **Order of Execution:** Base class constructor executes **first**, followed by the derived class constructor.

Question 4:

> **What is the correct way to overload the `<<` operator for a class Sensor? Justify.** *A) void operator<<(Sensor s); B) friend ostream& operator<<(ostream &out, const Sensor &s); C) Sensor operator<<(ostream &out); D) ostream operator<<(Sensor s); (CO1, Apply)*

- **Correct Answer: B) friend ostream& operator<<(ostream &out, const Sensor &s);**
- **Justification:** The left-hand operand of `<<` is `ostream` (`cout`), not our class. Therefore, it cannot be a member function and must be a **friend function**. It returns a reference to `ostream` (`ostream&`) to allow cascading (e.g., `cout << s1 << s2;`), and takes the sensor object by constant reference (`const Sensor &s`).

Question 5:

> **You are designing a sensor system on Raspberry Pi where a base class `Sensor` contains general attributes like `sensorID` and a method `readData()`. A derived class `TemperatureSensor` adds a method `readTemperature()`. Which of the following best demonstrates how to extend functionality correctly? Justify.** *A) Override `readData()` in TemperatureSensor to remove sensorID initialization. B) Declare `readTemperature()` as a friend function of Sensor. C) Derive TemperatureSensor from Sensor publicly and add `readTemperature()` method. D) Declare TemperatureSensor as a private member inside Sensor. (CO2, Apply)*

- **Correct Answer: C) Derive TemperatureSensor from Sensor publicly and add `readTemperature()` method.**
- **Justification:** Inheritance is designed to specialize and extend functionality. Public inheritance (`public Sensor`) preserves the "is-a" relationship, allowing `TemperatureSensor` to inherit base attributes (`sensorID`, `readData()`) while cleanly adding its own specialized behavior (`readTemperature()`).

Here are the answers and explanations for the next set of questions from your test paper:

Question 6.a.i:

> **What is the purpose of a copy constructor in C++? Justify.** *A) To initialize arrays B) To convert types C) To create a copy of an object D) To overload arithmetic operators (CO1, Understand, 2 Marks)*

- **Correct Answer: C) To create a copy of an object**

- **Justification:** A copy constructor is a special member function that initializes a new object using an existing object of the same class (e.g., `Class obj2 = obj1;`). It takes a constant reference to an object of the same class (`Class(const Class &obj)`).

Question 6.a.ii (Snippet 1):

> **What is the output of the following code? Justify.** *A) 15 B) 10 C) 5 D) Error (CO1, Apply, 2 Marks)*

- **Correct Answer: A) 15**
- **Justification:**
  - `Value v1(5)` sets `v1.x = 5`.
  - `Value v2(10)` sets `v2.x = 10`.
  - `v1 + v2` invokes `operator+`, adding `v1.x + v2.x` (`5 + 10 = 15`) and returns a new temporary object initialized with `15`.
  - `Value v3 = v1 + v2;` copies this result into `v3`.
  - Calling `v3.show()` prints `15`.

Question 6.a.iii:

> **Describe the concept of shallow and deep copying in C++. How does operator overloading help in implementing deep copy in user-defined classes?** *(CO1, Understand, 6 Marks)*

- **Direct Answer & Justification (Concise for 6 Marks):**
  1. **Shallow Copy:**
     - Copies member variables bit-by-bit. If a class has a *pointer*, it copies only the memory address. Both objects end up pointing to the exact same heap memory location, causing double-deletion crashes and data corruption.
  2. **Deep Copy:**
     - Allocates a brand-new block of memory on the heap for the new object and copies the *actual data values* over. Both objects remain completely independent.
  3. **Role of Operator Overloading (`operator=`):**
     - When you assign one object to another (`obj2 = obj1`), C++ defaults to a shallow copy. By **overloading the assignment operator (`=`)**, we can write custom logic to delete old memory, allocate fresh heap memory, and copy string/array data character-by-character, guaranteeing a safe **deep copy**.

Here are the answers and explanations for this next set of test questions:

Question 6.b.i:

> **What happens if you don't define a copy constructor and assignment operator in a class with dynamic memory? Justify.** *A) Code will not compile B) Compiler throws an error C) Deep copy will happen by default D) Shallow copy will be performed (CO1, Understand, 2 Marks)*

- **Correct Answer: D) Shallow copy will be performed**

- **Justification:** If a class manages dynamic memory (pointers) and you do not explicitly define your own copy constructor and assignment operator, the compiler automatically generates default ones which perform a **shallow copy**. This copies only the pointer address, leading to double-deletion crashes and memory corruption.

Question 6.b.ii:

> **What does the following code represent? Justify.**
>
> ```
> class Battery {
>     PowerCell cell; // PowerCell is another class
> };
> ```
>
> *A) Inheritance B) Aggregation C) Composition D) Association (CO1, Apply, 2 Marks)*

- **Correct Answer: C) Composition**
- **Justification:**
  - This is a "has-a" relationship where one class owns another as a member object directly inside it.
  - **Composition** represents strict ownership: if the `Battery` object is destroyed, the `PowerCell` object inside it is also destroyed. (Unlike aggregation, where the part can exist independently).

Question 6.b.iii:

> **Describe the role of converting constructors and explain how the `explicit` and `delete` keywords affect them in C++.** *(CO1, Understand, 6 Marks)*

- **Answer & Justification (Concise for 6 Marks):**
  1. **Converting Constructor:**
     - A constructor that can be called with a **single argument**. By default, C++ uses it for **implicit type conversion** (e.g., if you have `MyClass(int x)`, you can write `MyClass obj = 10;` and C++ will automatically convert the integer `10` into a `MyClass` object).
  2. **The `explicit` Keyword:**
     - Placed before a converting constructor (e.g., `explicit MyClass(int x);`) to **disable implicit conversions**. It forces the programmer to use explicit casting, preventing accidental bugs caused by hidden type conversions.
  3. **The `delete` Keyword:**
     - Used to explicitly **disable or ban** a constructor or function (e.g., `MyClass(int) = delete;`). If someone tries to pass an integer to initialize the class, the compiler throws an error, completely preventing that type of conversion/construction.
